

CSS3 Tutorial: Advanced Level

For people comfortable with custom properties, Grid, Flexbox, BEM, and specificity, who want to work at a production/architecture level.

Table of Contents

1. [@property : Typed Custom Properties](#)
2. [The :has\(\) Relational Selector](#)
3. [Cascade Layers with @layer](#)
4. [Modern Color: color-mix\(\) and oklch\(\)](#)
5. [Subgrid](#)
6. [Logical Properties for Internationalization](#)
7. [Native CSS Nesting](#)
8. [Scroll-Driven Animations](#)
9. [The View Transitions API](#)
10. [Feature Detection with @supports](#)
11. [Performance: content-visibility and will-change](#)
12. [Respecting User Preferences](#)
13. [Architecture at Scale](#)
14. [Practice Project: A Themeable, Accessible Component Library](#)
15. [Where to Go Next](#)

1. `@property` : Typed Custom Properties

Regular custom properties are just untyped strings, which means they can't be smoothly animated (a browser doesn't know how to interpolate between two arbitrary strings).

`@property` registers a custom property with a real type, so it can be transitioned like any other value:

```
@property --angle {  
  syntax: "<angle>";  
  initial-value: 0deg;  
  inherits: false;  
}  
  
.dial {  
  background: conic-gradient(from var(--angle),  
    red, orange, yellow);  
  transition: --angle 0.4s ease;  
}  
.dial:hover {  
  --angle: 180deg;  
}
```

Without `@property`, that gradient rotation would snap instantly instead of animating smoothly.

2. The `:has()` Relational Selector

`:has()` lets a selector match a **parent** based on its children — something CSS couldn't do before. It's often described as a "parent selector."

```

/* Style a form group only if it contains an
invalid input */
.form-group:has(input:invalid) {
  border-color: red;
}

/* Style a card differently if it contains an
image */
.card:has(img) {
  grid-template-columns: 120px 1fr;
}

/* Dim every card except the one being hovered
*/
.grid:has(.card:hover) .card:not(:hover) {
  opacity: 0.5;
}

```

This eliminates a lot of JavaScript that used to exist purely to toggle a class based on a child's state.

3. Cascade Layers with `@layer`

As stylesheets grow — especially with a mix of resets, a design system, third-party CSS, and utility overrides — specificity wars become common. `@layer` lets you define explicit priority **independent of specificity or source order**:

```

@layer reset, base, components, utilities;

@layer reset {
  * { margin: 0; padding: 0; box-sizing: border-box; }
}

```

```

}

@layer components {
  .button { padding: 8px 16px; border-radius:
4px; }
}

@layer utilities {
  .text-center { text-align: center !important;
}
}

```

Layers declared later in the top-level list always win over earlier ones, **regardless of specificity** — a `#id` selector in `reset` still loses to a `.class` selector in `utilities`. This is the modern fix for "I had to add `!important` because I couldn't beat this selector."

4. Modern Color

`color-mix()` blends two colors without manual math:

```

.button:hover {
  background: color-mix(in srgb, var(--brand-
color) 80%, black);
}

```

`oklch()` is a newer color space that's perceptually uniform — meaning if you change the lightness value, the color actually looks uniformly lighter/darker to the human eye (unlike HSL, which can shift perceived brightness unpredictably):

```
:root {  
  --brand: oklch(60% 0.15 250);  
}
```

Both pair well with `@supports` (below) since older browsers may not recognize them yet.

5. Subgrid

Normally, a nested grid has no awareness of its parent's rows/columns. `subgrid` lets a nested grid inherit the parent's tracks, so items across different cards can still align to a shared column/row structure:

```
.card-grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: auto auto auto;  
}  
  
.card {  
  display: grid;  
  grid-row: span 3;  
  grid-template-rows: subgrid; /* inherits the  
  parent's row sizing */  
}
```

This solves the classic "why don't all my card titles/descriptions/buttons line up across cards" problem without hacky fixed heights.

6. Logical Properties for Internationalization

Physical properties (`margin-left` , `padding-right`) assume left-to-right text. Logical properties adapt automatically to the page's writing direction:

```
.card {  
  margin-inline-start: 16px; /* "left" in LTR,  
  "right" in RTL */  
  padding-block: 12px;      /* top and bottom  
  */  
  border-inline-end: 1px solid #ddd;  
}
```

If a project ever needs to support right-to-left languages (Arabic, Hebrew), logical properties mean the layout flips correctly with zero extra CSS.

7. Native CSS Nesting

Previously only available via Sass, nesting is now native CSS:

```
.card {  
  padding: 16px;  
  border-radius: 8px;  
  
  & .title {  
    font-size: 1.25rem;  
  }  
}
```

```
&:hover {  
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);  
}  
  
@media (max-width: 480px) {  
  padding: 8px;  
}  
}
```

The `&` refers to the parent selector, same convention as Sass. This reduces repetition without needing a build step or preprocessor.

8. Scroll-Driven Animations

Animations can now be tied directly to scroll position instead of time, with no JavaScript scroll listener required:

```
@keyframes fade-in {  
  from { opacity: 0; transform:  
translateY(20px); }  
  to   { opacity: 1; transform: translateY(0); }  
}  
  
.reveal {  
  animation: fade-in linear;  
  animation-timeline: view();  
  animation-range: entry 0% cover 40%;  
}
```

This drives the classic "fade in as you scroll" effect, previously requiring an `IntersectionObserver` and manual class toggling, purely in CSS.

9. The View Transitions API

Lets you animate between two DOM states (or even two separate page loads) with a smooth cross-fade/morph, again without hand-built JavaScript animation logic:

```
::view-transition-old(root),  
::view-transition-new(root) {  
  animation-duration: 0.4s;  
}
```

```
document.startViewTransition(() => {  
  // update the DOM here, e.g. render a new page  
  section  
  updateContent();  
});
```

The browser automatically captures a snapshot before and after and animates between them.

10. Feature Detection with @supports

Since several features above are newer, `@supports` lets you ship a fallback safely:

```
.dial {  
  background: conic-gradient(red, orange,  
    yellow); /* fallback */  
}
```



```
@supports (background: conic-gradient(from 0deg,
red, orange)) {
  .dial {
    background: conic-gradient(from var(--angle,
0deg), red, orange, yellow);
  }
}
```

Write the safe, widely-supported version first as the default, then progressively enhance inside `@supports` blocks — this is the standard pattern for adopting new CSS features in production.

11. Performance

`content-visibility: auto` tells the browser to skip rendering work for off-screen content until it's needed — a major win on long pages:

```
.article-section {
  content-visibility: auto;
  contain-intrinsic-size: 0 500px; /* estimated
size so scrollbars don't jump */
}
```

`will-change` hints to the browser that a property is about to animate, letting it optimize ahead of time — but use it sparingly and only right before an animation starts, since overuse consumes extra memory:

```
.modal {
  will-change: transform, opacity;
```

```
}
```

12. Respecting User Preferences

Production CSS should respond to system-level accessibility preferences:

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation: none !important;  
    transition: none !important;  
  }  
}  
  
@media (prefers-color-scheme: dark) {  
  :root {  
    --bg: #111;  
    --text: #eee;  
  }  
}  
  
@media (prefers-contrast: more) {  
  .button {  
    border: 2px solid currentColor;  
  }  
}
```

Skipping `prefers-reduced-motion` in particular can genuinely cause discomfort (nausea, vertigo) for some users — treat it as a baseline requirement, not a nice-to-have.

13. Architecture at Scale

At this level, the main challenge shifts from individual properties to keeping a large stylesheet maintainable:

- **Layer your cascade** (`@layer reset, base, components, utilities`) so priority is explicit.
 - **Centralize design tokens** as custom properties (`--space-1` through `--space-8` , a type scale, a color palette) rather than magic numbers scattered through files.
 - **Co-locate component styles** with their markup/component files rather than one giant global stylesheet.
 - **Lint for specificity creep** — if you're regularly reaching for `!important` , it's usually a sign the cascade layering or naming convention needs revisiting, not that `!important` is the right tool.
-

14. Practice Project

Build a small themeable component library:

1. Define a full set of design tokens (color, spacing, radius, type scale) as custom properties in `:root` , plus a `[data-theme="dark"]` override using `color-mix()` or `oklch()` .
2. Structure your CSS with `@layer tokens, reset, components, utilities` .

3. Build a card component using **subgrid** so titles and footers align across a row of cards.
 4. Add a `:has()` rule so a form's submit button is disabled-looking whenever any required field is `:invalid`.
 5. Add a scroll-driven fade-in for cards entering the viewport, wrapped in a `@media (prefers-reduced-motion: no-preference)` guard.
-

15. Where to Go Next

- **CSS Houdini** (Paint API, Properties & Values API) for writing custom rendering logic
 - **Design tokens tooling** (Style Dictionary or similar) to generate CSS variables from a single source of truth shared with design tools
 - **CSS-in-JS vs. vanilla CSS trade-offs** if working in component-heavy frameworks — understand what problem each approach solves before choosing
 - Browser support tables (caniuse.com) as a habit before shipping any feature from this tutorial to production
-

Tip: at the advanced level, the skill isn't knowing every new CSS feature — it's knowing which one solves the actual problem in front of you, and shipping it with a safe fallback.